CEG4158 - Computer Control in Robotics

Robotics Laboratory Guide

Lab #1

Robotic Manipulator Exploration and Simulation

A. Valencia, P. Payeur, F. Nadon

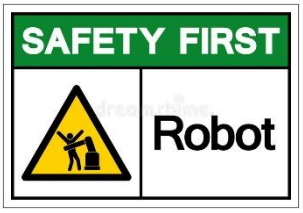
Fall 2026

School of Electrical Engineering and Computer Science

University of Ottawa

©2026

This material is protected by copyright. Reproduction, distribution, modification, or uploading to any platform of this document are strictly prohibited.



WARNING! Always play safe with robots.

1 Introduction

This first laboratory aims to introduce the Gen3 lite robot developed by Kinova, a Canadian company. The Gen3 lite is a 6 revolute degrees-of-freedom manipulator (6R) equipped with a two-finger gripper. It allows the manipulation of objects up to a weight of 500 grams within a workspace of up to 76 cm. The user manual "*Gen3_lite_USER_GUIDE_release2021.pdf*" available on Brightspace provides a large amount of information about the characteristics and the operation of the robot. The illustration on page 13 of the guide details the main components of the manipulator. This robot can be controlled in different ways including via an Xbox remote control, via the Kortex Web App that exploits a mini web server encoded in the robot's controller, via the Matlab Wrapper which is a Matlab toolbox that provides advance functionalities to program the manipulator, or via the application programming interface (API) KINOVA KORTEX developed by the manufacturer. You will be given the opportunity to briefly (~30 min. over the lab. #1 sessions) explore the manipulator's operation using the Kortex Web App. For the remaining of the work, a simulator built over CoppeliaSim Edu v4.7.0 will be used to develop your own solutions and run experiments, for safety and group size reasons.

CoppeliaSim Edu (formerly known as V-REP) is a powerful multi-platform robotic simulator, mainly open-source and that offers a free educational license. It provides a large variety of pre-built robots, sensors and objects, and supports up to six different programming languages. All these features make CoppeliaSim a very versatile simulator among others. The documentation is extremely helpful and comprehensive. You may consult online CoppeliaSim manual and tutorials to learn more. The forum is another great resource for finding solutions to common problems.

This guide explains the steps required to operate simulations of the Kinova Gen3 lite robot and hand in CoppeliaSim Edu v4.7.0 from MATLAB using a simAPI programming tool. The simAPI is a custom software package that was developed at uOttawa to support lab work in the CEG4158 course by providing a simple interface between MATLAB and Kinova robots in both physical and simulated forms. Here you will find instructions on how to access and configure the tool, and on the use of commands to control the simulator, accompanied by several examples. While guiding you through the first lab protocol, the purpose of this guide is also to serve as a general reference document for all tasks that will be defined specifically for lab sessions in the CEG4158 course.

2 Objectives

This first laboratory session aims to introduce you to the Kinova Gen3 lite manipulator and to the CoppeliaSim Edu version 4.7.0 of the robot simulator to which you have local access on the computers in STE-3040 laboratory. The specific objectives includes:

1. Access the laboratory resources;
2. Operate a Kinova Gen3 lite manipulator via a simple graphical interface and examine the different movement capabilities of the robot;
3. Control a CoppeliaSim simulation from MATLAB using the simAPI developed for the purpose of this course;
4. Operate the joints of a simulated robot manipulator;
5. Closely observe the mechanical behavior of a manipulator arm and understand the relationship between joint-based control and the movements of a complex robotic architecture;
6. Expose participants to the challenges encountered when controlling a robotic manipulator;
7. Become familiar with programming functionalities used on a commercial robotic platform.

3 Important notes

- Lab sessions will take place **in person** à STE-3040 in accordance with the course schedule.
- Students should perform the lab procedures **in teams of two** given the limited number of workstations available compared to the number of enrolled participants in the course. Collaboration is permitted and encouraged. However, evaluation must be performed individually for every team. Lab grades will be team based.
- Only the STE-3040 workstations are configured with the simulation software. Its use is therefore limited to the scheduled lab sessions (unless you opt to configure the free version of CoppeliaSim Edu v.4.7.0 on your own computer. Be aware that only version 4.7.0 is compatible with the provided simAPI). However, much of the preparation work can and should be performed outside the lab. **You are strongly encouraged to conduct the preparatory work before your next scheduled lab session**.

4 Exploring the physical Gen3 lite manipulator

Given the limited number of physical robots available in the laboratory, this part of the procedure will be performed on a time-shared basis among groups in your lab period. That is, a schedule will be set at the beginning of the first session for Lab #1. Make sure to be present to reserve your timeslot. Every team will be allocated a

period of 30 minutes to interact with one of the physical manipulators through Kinova's Kortex Web App. Timeslots will be allocated either for the 1st or 2nd session of Lab #1, which will allow all teams to become familiar with the real robot.

4.1 Accessing the Kortex Web App

The Kinova's Kortex Web App is a web-based application embedded in the physical robot controller which is contained in the base of the manipulateur.

1. Before attempting to connect to the manipulator, make sure that a USB cable connects the mini-USB port on the base of the robot to a USB-A port on the computer;
2. Power up the physical Gen3 lite robot with the switch on the base of the robot and wait for the green light before launching the Kortex Web App on the computer.
3. Open a web browser (Chrome recommended) and access the following IP address directly: 192.168.1.10. Ask the TA to provide the Username and Password information. They will not be shared to safety reasons.

4.2 Operating the robot with the native interface

1. Under the section on `Actuators`: Observe the various parameters that are displayed on the graphical interface of the Web App (see Figure 1) that represent the current state of the manipulator. Depending on the physical configuration in which the manipulator is currently set, the row of `Position` values for each of the six actuators will vary. Try to make sense of the six angular values that are displayed in correspondence with the respective alignment of the rigid parts that form the robot.

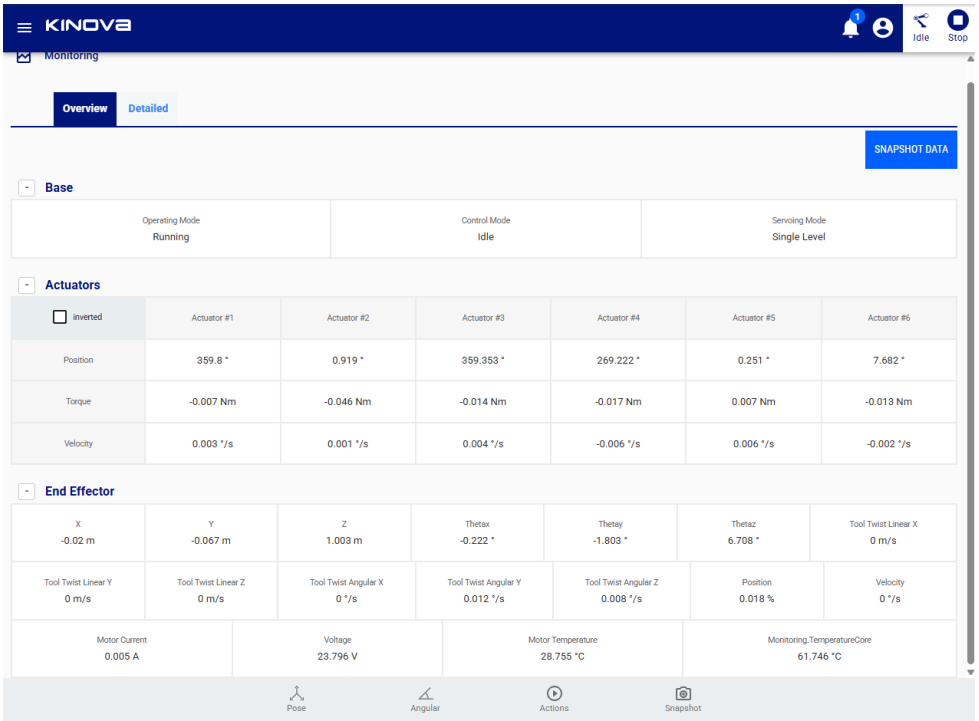


Figure 1: Kinova Kortex Web App – main interface

2. Under the section on `End Effector`: Notice the main `X, Y, Z` values that correspond to the gripper's location in 3D space with respect to a fixed reference frame attached to the base of the robot. The `Z` value is easier to interpret as it relates to the vertical elevation of the gripper with respect to the base.
3. Explore the functionalities under the set of buttons located at the bottom of the main screen. Begin with the `Angular` menu which is more intuitive. Manipulate the sliders to operate each one of the actuators, one at a time, to modify the configuration of the robot. Test each one of the actuators and for each of them carefully note the direction of the rotation of the corresponding joint when you respectively increase or decrease the angular value. This information will be crucial when you will model the manipulator in Lab #2 in order to eventually design its controller. Record this information in your notes and keep it preciously until Lab #2. Also open and close the gripper with the additional slider at the bottom of the joints control interface. This slider displays the percentage of closure of the gripper.

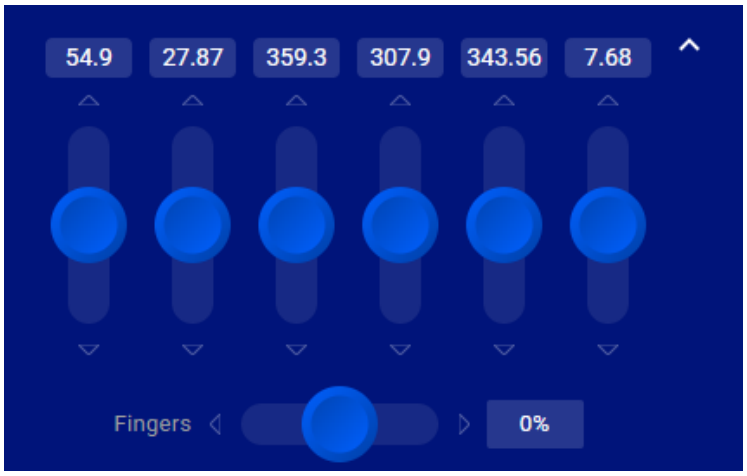


Figure 2: Kinova Kortex Web App – angular joints and gripper control

4. Explore an alternative mode to move the manipulator under the `Pose` menu at the bottom of the main interface. This control mode allows moving the end effector in the `Linear` space, that is along the `X, Y, Z` axes; or in the `Angular` space, that is around the `X, Y, Z` axes. While the manipulator moves, observe how the angular values change in the `Position` row of the main interface for each actuator. The relationship between the actuator`s angular values and the end effector position and orientation may not be straightforward. This is actually why you are taking this course. An additional slider opens or closes the gripper, with the display representing the percentage of closure.



Figure 3: Kinova Kortex Web App – end effector's position and orientation control

5. Finally, explore the demo sequence (`Demo`) and the pre-programmed configurations (`Home`, `Packaging`, `Retract`, `Zero`) available under the `Actions` menu at the bottom of the main interface. To bring the robot to one of the preset configurations, simply select the desired configuration from the drop-down list, then hit and hold the "play" button (small arrow) on the right side of the menu until the robot will reach the given configuration. Carefully examine what each configuration corresponds to. Some of them will be helpful for developing your forward kinematic model of the manipulator in Lab #2.
6. WARNING: Once you are done with exploring the physical manipulator, please do not leave the robot in a self-suspended configuration before closing the Web App application. Turning off the power may make the robot slowly collapse under its weight which may lead to damages, and significant cost to you…!
7. Close the gripper and bring the manipulator to a stable configuration where it won't collapse under its weight. You may use the `Retract` preset configuration. Then conclude the physical robot exploration by stopping the Kinova Web App with the `Stop` button on the top right corner of the main interface. Close the web browser.

5 Getting started with simulation

5.1 Accessing software programs

1. In order to access all MATLAB functionalities that will be used in the simulations, MATLAB R2024b that is installed on workstations at STE-3040 requires that you first add a mini-compiler on the local workstation. Perform the following procedure before launching Matlab:
   - Use Windows File Explorer and reach *`G:\pub\Courses\CEG4158\Matlab`*.
   - Run the file `Install_MinGW64_AddOn.cmd`. This will add the missing compiler to Matlab. A new window will open during installation. It may

take a few minutes. Just wait that installation completes when the window will close automatically.

- *Note:* This procedure must be performed at the beginning of every lab session because the Add-On gets erased at logout.

2. Start MATLAB on the local workstation at STE-3040:
   `Start Menu > MATLAB R2024b > MATLAB R2024b.`
3. Start CoppeliaSim Edu on the workstation at STE-3040.
   `Start Menu > CoppeliaSim Edu`

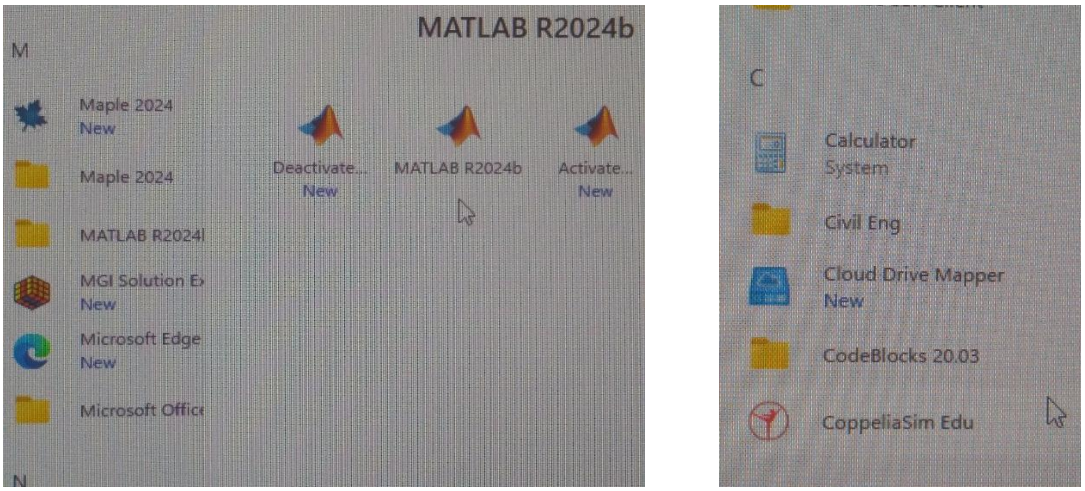


Figure 4: Launching Matlab and CoppeliaSim Edu from Windows Start Menu

Note: CoppeliaSim opens in full screen mode by default. You may want to make it a smaller size window over the desktop in order to also interact with MATLAB.

5.2 Configuration

On Matlab, you need to add the path where the simAPI files are located in order to use this tool. The default path is:

```
G:\pub\Courses\CEG4158\Matlab
```

This procedure must be performed for every new MATLAB session.

1. Go to the MATLAB window.
2. Add the simAPI path (defined above). It can be done in two ways:
   - Click on `Home > Set Path > Add with Subfolders ...` select the simAPI path, and click Save, and then Close. or
   - Use the command

     ```
     addpath(genpath('G:\pub\Courses\CEG4158\Matlab'));
     ```

     either in the MATLAB command window or at the very beginning of your script program in the MATLAB language.

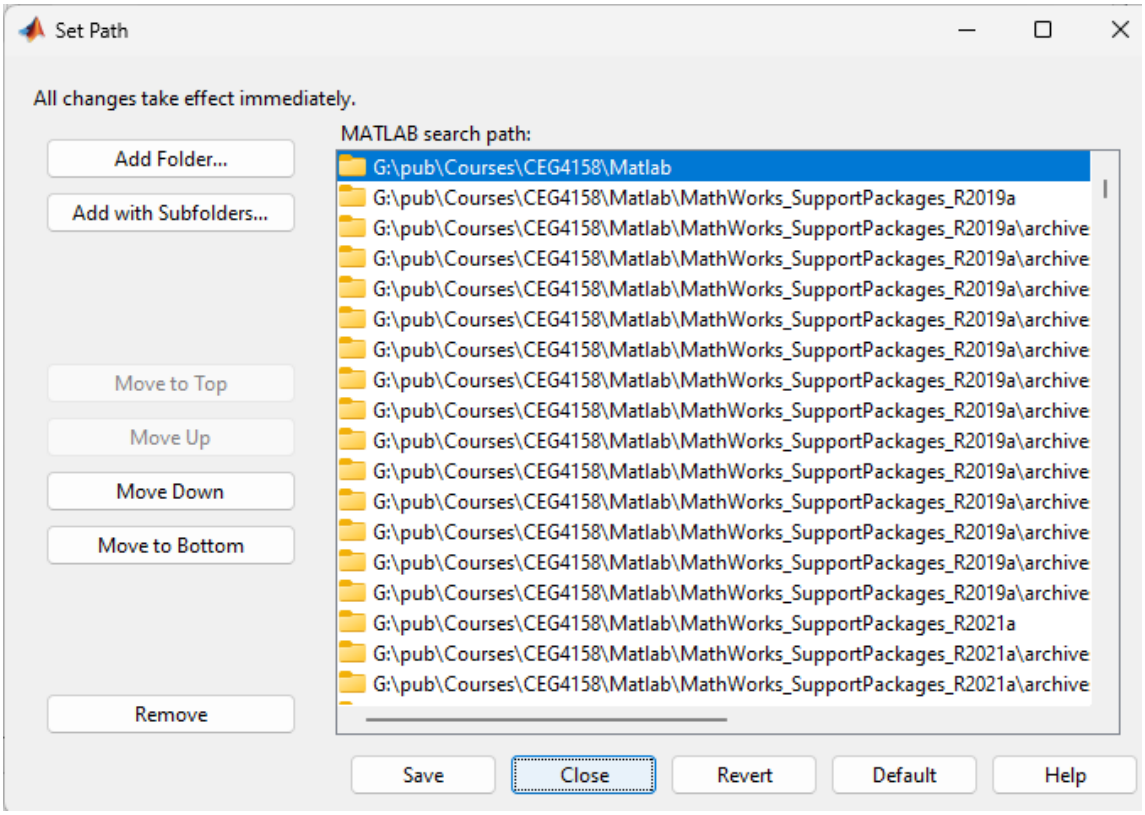


Figure 5: simAPI path added to MATLAB search path

3. In MATLAB, using the open folder icon with a green arrow just left of the path definition in the upper part of MATLAB window (see Figure 6) set your current session folder to:

   `G:\pub\Courses\CEG4158\Matlab`

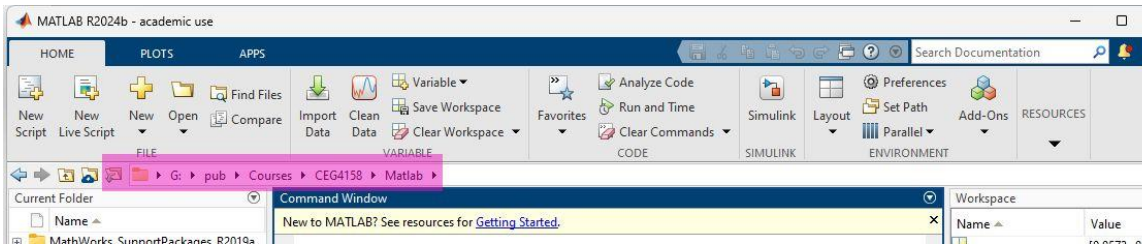


Figure 6: Setting the current session folder on MATLAB

Examples of how to use the simAPI to perform different simulation tasks are presented below. The steps required to replicate these examples are listed in each section, accompanied by the code and detailed explanation.

6 Basic simulation

This section explains the basic operation of a CoppeliaSim simulation using the VREP methods of the simAPI. Particularly, the content is limited to showing the minimal building blocks required for every simulation, which includes details on how to establish and terminate a communication between MATLAB and CoppeliaSim, start and stop the simulation, and open and close a virtual scene.

These are the essential steps on which other examples are built:

1. Create a VREP instance and start communication with the simulator
2. Open a scene
3. Start a simulation
4. Stop a simulation
5. Close a scene
6. End communication with simulator and destroy the VREP instance

The MATLAB program that runs this example is detailed here.

```
v = VREP('path','write CoppeliaSim path here')
v.loadscene('trajectoryAndMotion/trajectoryGenerationDemo')
v.simstart()
v.simstop()
v.clearscene()
v.delete()
```

6.1 Create a VREP instance

To start communication and establish a connection between MATLAB and CoppeliaSim, you need to create an instance of the VREP class, which we call `v` as an example, using the `VREP()` method contained in the simAPI. This method has as argument the CoppeliaSim installation path. The default path is:

```
C:\ProgramData\Microsoft\AppV\Client\Integration\6BF10E29-DF4B-4E33-
AEDF-FB25B6B1B588\Root\VFS\ProgramFilesX64\
CoppeliaRobotics\CoppeliaSimEdu
```

Make sure to leave no empty space in the path definition when you copy it in your script file. It must be a continuous string of characters.

```
v = VREP('path','write CoppeliaSim path here')
```

```
Command Window
>> v = VREP('path','c:\ProgramData\App-V\9C55BA6E-D98C-4916-927C-4F616A7B9A71\9B6A50D8-E896-4CB1-BC47-F
Note: always make sure you use the corresponding remoteApi library
(i.e. 32bit Matlab will not work with 64bit remoteApi, and vice-versa)

v =
V-REP robotic simulator interface (active=1)
path: c:\ProgramData\App-V\9C55BA6E-D98C-4916-927C-4F616A7B9A71\9B6A50D8-E896-4CB1-BC47-F4BF892EC48D\Ro
mode: simx_opmode_oneshot_wait (blocking)
fx >>
```

Figure 7: VREP instance creation (using an older CoppeliaSim path)

6.2 Open a scene

Once connection is established, proceed to open a predefined virtual scene. This can be done by calling the `loadscene()` method on the VREP instance `v` we just created. You need to pass the name of the scene you want to load. The method will search for the scene in the default folder `\scenes` within CoppeliaSim path.

```
v.loadscene('trajectoryAndMotion/trajectoryGenerationDemo')
```

Go to CoppeliaSim window and you should see a scene containing one manipulator with three containers holding green and red parts, as shown in Figure 8. Move the subwindow displaying the velocity and acceleration parameters to the left so that you can appreciate the entire simulated scene. This scene is predefined in the CoppeliaSim package installed on the lab workstations, to serve as an example and demonstration. Using the `loadscene()` method, you can of course open and explore any scene you want, assuming that the file exists and its location is correctly specified.

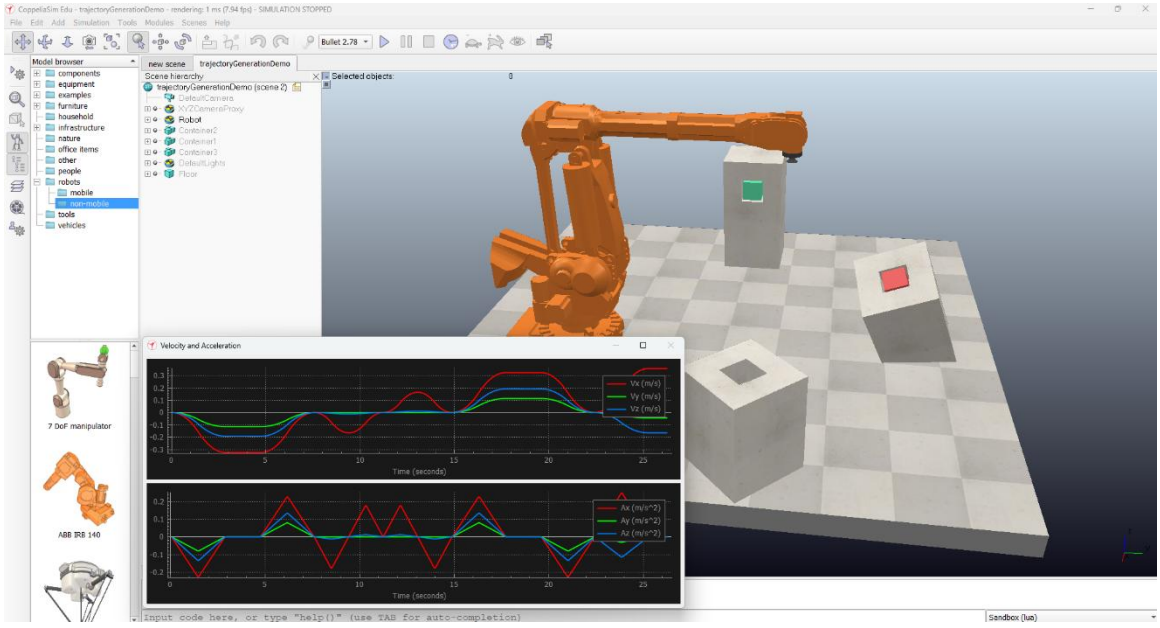


Figure 8: Loaded a predefined CoppeliaSim virtual scene

Note: This command has the same effect as if the scene were loaded in CoppeliaSim window by clicking on `File > Open scene...` and searching for the `.ttt` file in the appropriate folders.

6.3 Start simulation

Now is time to run the simulation. To do this, simply call the `simstart()` method on the VREP instance `v`.

```
v.simstart()
```

Quickly go to the CoppeliaSim window and observe what task is performed in the simulator. If you loaded the same scene as the one in Figure 8, you should see a pick-and-place task with the robotic arm exchanging two colored blocks in between the three containers. It is possible that you observe variations of this scenario due to effects of the physics engine that influences the behavior of the objects while the simulation runs. This is only a demonstration of the simulator's potential.

6.4 Stop simulation

Once the simulation has run for some time, you need to stop it. This can be done by using the `simstop()` method on the VREP instance `v`. This will return the scene back to its original state.

```
v.simstop()
```

6.5 Close a scene

Once you finish using this scene, it may be a good idea to close it. To do this, you can use the `clearscene()` method on the VREP instance `v`. By default, this command closes the active scene in CoppeliaSim. If you have several scenes open, you need to pass the name of the scene to close as an argument, similar to how you used the `loadscene()` command above.

```
v.clearscene()
```

Note: Beware of this command, as it immediately closes the scene regardless of whether the scene is saved to disk or not. Also, it will not close the scene if you have not stopped the simulation first with the `simstop()` method.

6.6 Destroy the VREP instance

As a good practice, proceed to terminate the connection and communication between MATLAB and CoppeliaSim. You need to call the `delete()` method on the VREP instance `v`. This method also destroys the instance in the process.

```
v.delete()
```



Figure 9: VREP instance destruction

7 Robot simulation

In the context of the CEG4158 course, we will consider the Kinova Gen3 lite robot as our experimental platform. This manipulator robot comes with 6 degrees of freedom (DoF) and a two-finger gripper. A detailed presentation of the robot is available in the manufacturer's User Guide "*Gen3_lite_USER_GUIDE_release2021.pdf*" documentation available on Brightspace, under "Labs". You may want to examine the figure on page 13 of the User Guide to make yourself familiar with the robot`s architecture. When you allocated timeslot to explore the physical robot comes during Lab #1, follow the procedure detailed in Section 4 (above) and closely observe how the physical robot moves each of its joints, as well as the effect of combining joint movements on the 3D change of position and orientation of the gripper.

This section explains how to operate the Gen3 lite robot simulator using the GEN3LITE methods contained in the simAPI.

Examples are presented in three parts. Sections 7.1 and 7.2 show the most basic operations involving the Gen3 lite robot. Section 7.3 shows how to manually operate the robot with a graphical user interface (GUI), and Section 7.4 shows the robot operation using some motion commands that allow you to develop programmed scripts to drive the robot.

As part of Sections 7.3 and 7.4, you will also be invited to perform some experimental activities to develop your skills with operating the robot.

7.1 Initialization

This subsection explains how to load a virtual model of the Gen3 lite robot and how to connect to and disconnect from the robot.

1. Create a VREP instance and start communication with the simulator
2. **Load a robot model**
3. Start a simulation
4. **Create a Gen3Lite instance**
5. **Connect to the robot**
6. **Disconnect the robot**
7. Stop the simulation
8. Close a scene
9. End communication with simulator and destroy the VREP instance

The MATLAB program that runs this example is detailed here.

```
v = VREP('path','write CoppeliaSim path here')
v.loadmodel('Gen3Lite robot elevated','local',true)
v.simstart()
j = GEN3LITE(v,'Gen3Lite')
j.connect()
j.disconnect()
v.simstop()
v.clearscene()
v.delete()
```

The above steps show that before operating the robot in CoppeliaSim, you need to first establish communication between MATLAB and the simulator by creating a VREP instance. The initial steps (1 and 3) are very similar to the example in Section 6. The same applies for the last steps (7-9). The new steps (2, 4-6) are discussed in more detail below.

7.1.1 Load a robot model

In Section 6.2, you learned how to open a scene, which is nothing more than a collection of models (these are listed in the `Scene hierarchy` of the CoppeliaSim interface). Loading a model is very similar, you simply need to use the `loadmodel()` method on the VREP instance `v`, which expects the name of the model as an argument. This method will search for the model in the default folder `\models` within the simAPI path defined in Section 5.2.

For this example, load the model named `Gen3Lite robot elevated`. This is a custom model built to match the type of robot available in the lab. The elevation component in the model captures the thickness of the attachment plate that surelevates the entire robot by 52 mm over the table. The model is also designed to communicate with the simAPI. All GEN3LITE methods contained in the simAPI expect this model to be present in the scene, otherwise they will fail.

```
v.loadmodel('Gen3Lite robot elevated','local',true)
```

Note that the optional argument `'local'` is set to true to let the method search for the custom model in the folder \models within the local simAPI path and not in the CoppeliaSim path, which is the default behavior, as shown in Section 6.2.

Go to CoppeliaSim and you should see the custom Gen3Lite model loaded into an empty scene, as shown in Figure 10.

Note: This command has the same effect as if the model was loaded in CoppeliaSim by clicking on `File > Load model...` and searching for the `.ttm` file in the appropriate folders.

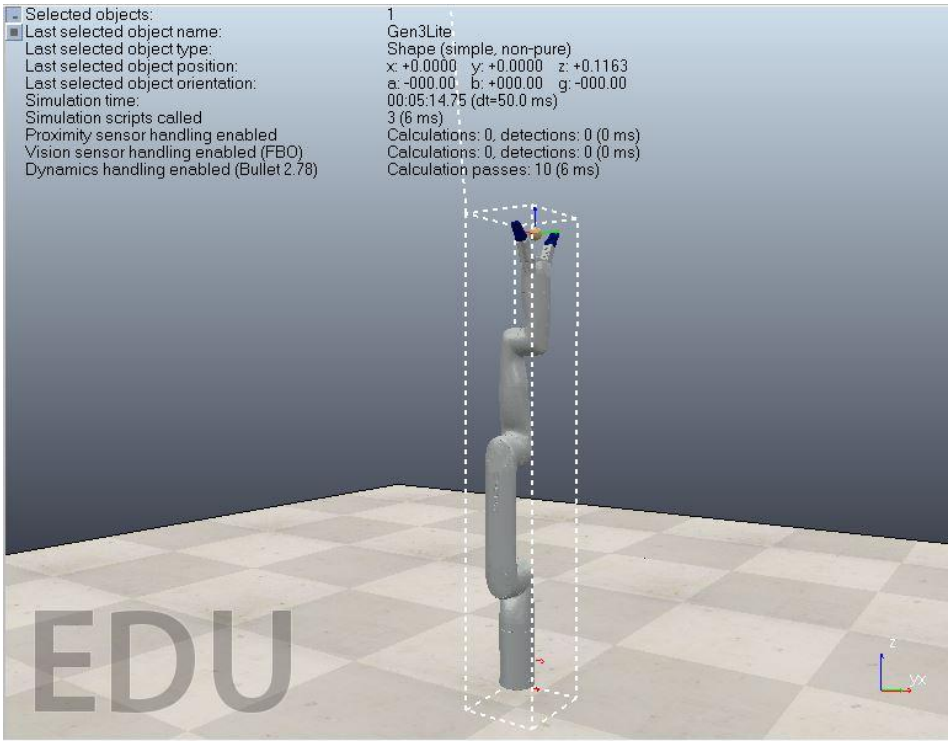


Figure 10: 6-DoF Gen3 lite model (`Gen3Lite robot elevated.ttm`)

7.2 Create a Gen3Lite instance

After you start the simulation with the `simstart()` method and in order to support the operation of the simulated robot from commands generated through the simAPI, you need to create an instance of the Gen3Lite class, which we call `j` as an example, using the `GEN3LITE()` method. This method has as arguments: a VREP instance and the name of the robot in the scene, which in our example corresponds to the VREP instance named `v`, and robot named `Gen3Lite`.

```
j = GEN3LITE(v,'Gen3Lite')
```

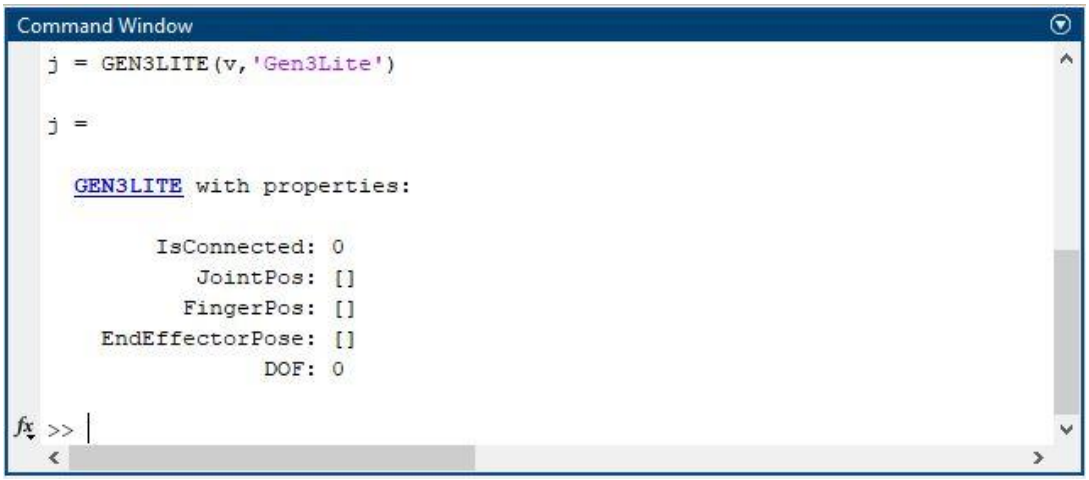


Figure 11: Gen3Lite instance creation

7.2.1 Connect to the robot

Next, a connection to the robot is required using the `connect()` method on the Gen3Lite instance `j` to enable the operation of the robot. The connection property is disabled by default, so it does not return any status value for the robot state, as shown in Figure 12.

```
j.connect()
```



Figure 12: Successful robot connection

7.2.2 Disconnect the robot

After simulation, the robot should be disconnected to prevent any unwanted operation. You have to call the `disconnect()` method on the Gen3Lite instance `j` to proceed to disconnect the service to the robot.

```
j.disconnect()
```



Figure 13: Successful robot disconnection

7.3 Graphical User Interface (GUI) operation

This subsection explains how to operate the arm joints and fingers of the Gen3 lite robot using a graphical interface (GUI).

1. Create a VREP instance and start communication with the simulator
2. Load a robot model
3. Start a simulation
4. Create a Gen3Lite instance
5. Connect to the robot
6. **Launch the GUI and command the joints with sliders**
7. Disconnect the robot
8. Stop simulation
9. Close a scene
10. End communication with simulator and destroy the VREP instance

The MATLAB program that runs this example is detailed here.

```
v = VREP('path','write CoppeliaSim path here')
v.loadmodel('Gen3Lite robot elevated','local',true)
v.simstart()
j = GEN3LITE(v,'Gen3Lite')
j.connect()
j.GUI()
j.disconnect()
v.simstop()
v.clearscene()
v.delete()
```

This program is based on the one in Section 7.1. The new step (6) is discussed in more detail below.

7.3.1 Launch the GUI and command the joints

The GUI allows individual position control of the robot's 6 joints and pair of fingers. It emulates the operation of a virtual joystick, a feature commonly found in industrial robots, and known as the "teach pendant". Simply call the `GUI()` method on the Gen3Lite instance `j` to launch this graphical user interface.

```
j.GUI()
```

Joints and fingers of the Gen3 lite robot can be moved by manipulating the appropriate sliders or changing the numerical values in the input and display fields on the right of each slider (double click in the window and type in a value), as shown in Figure 15.

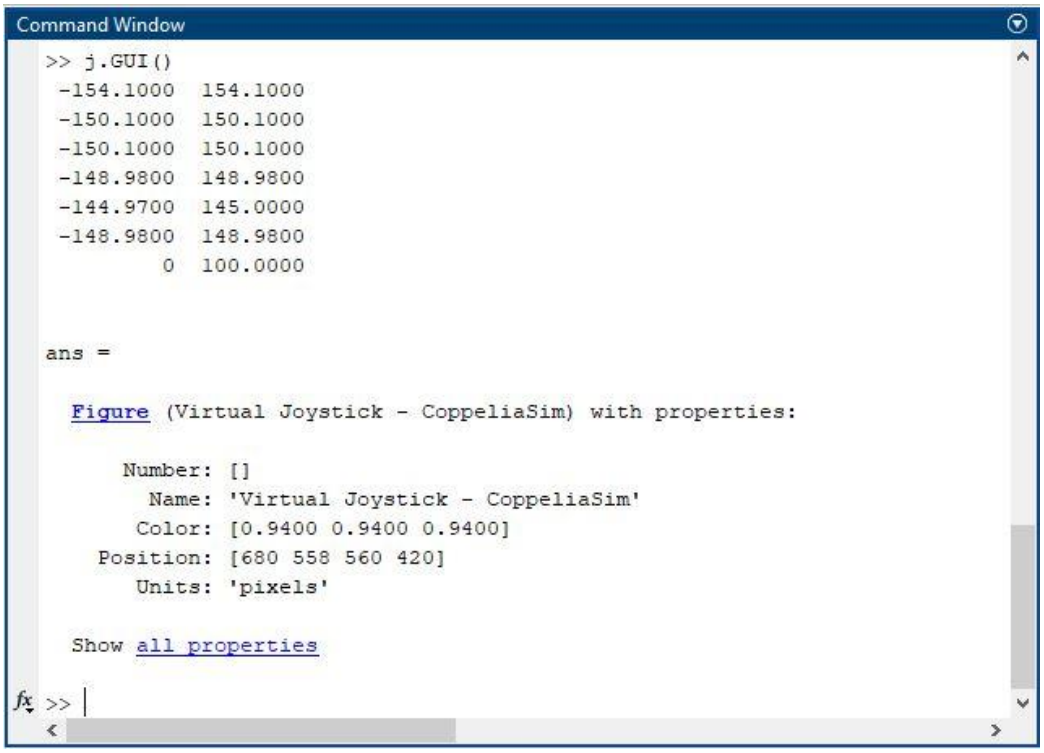


Figure 14: Successful GUI launch

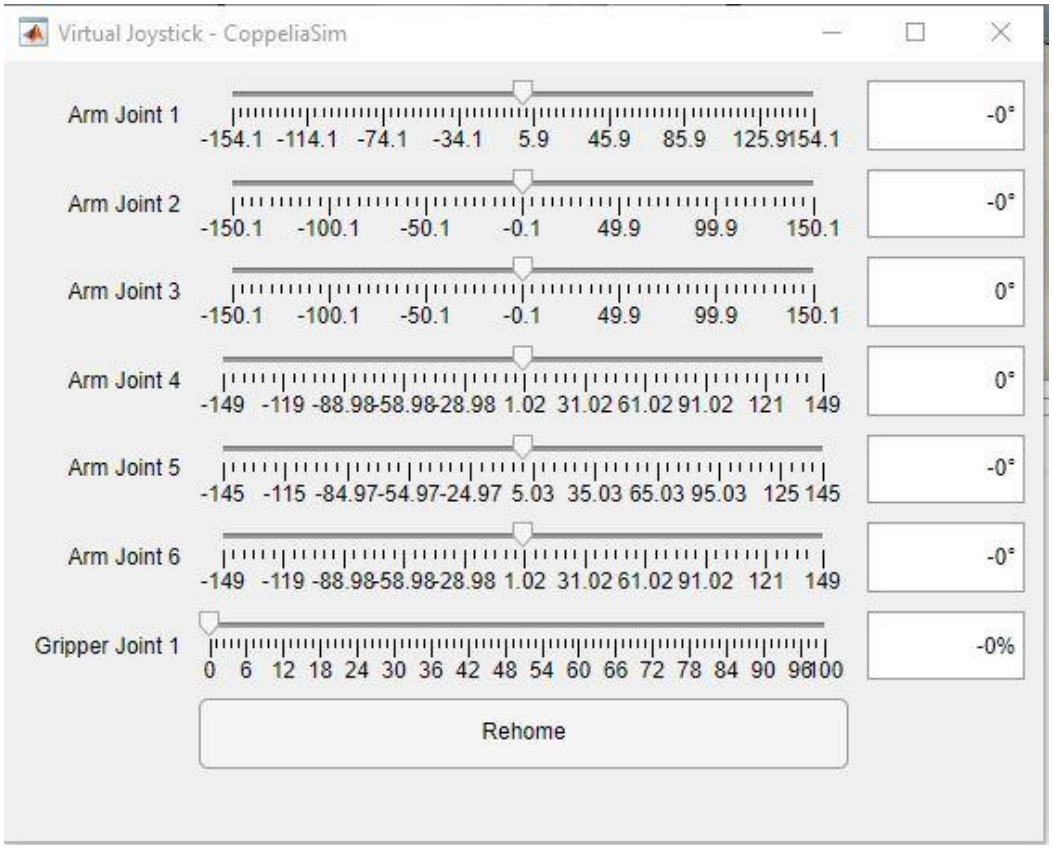


Figure 15: GUI layout

The arm contains six joints, numbered in sequence from the base joint (`Arm Joint 1`) to the extremity of the arm (`Arm Joint 6`). For the Hand, the fingers are underactuated, that is a single control parameter (`Gripper Joint 1`) drives both fingers, making the hand open and close symmetrically. The different sections of each finger are mechanically articulated and do not require a specific control parameter.

The `Rehome` button at the bottom of the GUI moves the robot to its "home" all straight up configuration (called "zero" in Kinova's documentation and Kortex Web App). It will also restart the GUI in the process, so wait a moment for the GUI to reappear.

Note that the joint values displayed in the GUI are in degrees. This is simply for visualization purposes, as all GEN3LITE methods that handle joints expect angular values in radians, as will be seen in Section 7.4.

Pay attention to the range of angular values available for each joint, as displayed by the minimum and maximum angles on each slider. If you type in values in the input field that are beyond those limits, the simulator will stop at the limit and not fully respond to your command.

The ranges are adjusted according to the physical robot limits, which are summarized in Table 1. You can consult the manufacturer's User Guide "*Gen3_lite_USER_GUIDE_release2021.pdf*" on pages 71-72 for a more comprehensive analysis of joint limitations on the 6 DoF Gen3 lite robot.

Table 1: Kinova Gen3 lite joint limits

Joint i	Min(°)	Max(°)
1	-154.1	154.1
2	-150.1	150.1
3	-150.1	150.1
4	-148.98	148.98
5	-144.97	145.0
6	-148.98	148.98

7.3.2 Experimental activities using the GUI

Perform the following activities to become familiar with the Gen3 lite robot architecture, the simulator and controlling movement via the GUI.

1. Using the `Gen3Lite robot elevated` model, observe the by-default configuration of the robot upon opening the GUI. Note the values of all joint angles and closely observe the corresponding configuration of the manipulator. This configuration will be referred to as the "Home Configuration".

2. Manipulate the sliders one joint at a time, beginning from Arm `Joint 1` and until Arm `Joint 6`. Proceed slowly and take time to set the slider corresponding to a given joint to different values and observe the movement performed by the

robot. Pay close attention to the direction of the movement and its relative orientation in 3D space as you increase or decrease each joint angle. Also examine the behavior close to the joint limits.

3. Perform a similar joint-by-joint analysis of the robot behavior using the input fields on the right of each slider. Observe the relative accuracy that you can reach in comparison with using only the sliders to command the manipulator.

4. Return the manipulator to its "Home Configuration" by setting all 6 Arm Joints values to 0° or using the `Rehome` button. Then using the mouse and the camera pan-rotate-shift buttons on the upper left corner of CoppeliaSim's interface, change the virtual observer viewpoint in the simulator until you zoom in on the two-finger robotic hand attached to the robot. Position the observer to be as perpendicular as possible to the two fingers (i.e., a side view with the two fingers in evidence on the left and right sides of the display). Then, using the slider or input field in the GUI, study the behavior of the fingers as you vary the value for `Gripper Joint 1`. Notice the symmetry of displacement of the two fingers and the values that lead to the gripper being fully open or fully closed.

5. Bring the virtual observer back to a location that lets you observe the entire robot behavior from a lateral perspective. Attempt various configurations of the manipulator using combinations of "simple" angles on `Arm Joint 1` to `Arm Joint 6`, using either the sliders or the input fields in the GUI (e.g., set all joints to 90°, to 45°, or to 135°, or combinations of such values). Observe how these joint angle combinations result in different configurations of the robot arm.

6. Using the simAPI methods that you learned about in the previous sections, disconnect the `Gen3Lite robot elevated`, stop the simulation, and close the simulator scene. Then load the `Gen3Lite blocks` scene instead, which contains the same manipulator model but with additional virtual objects, as shown in Figure 16. Use the `loadscene()` command. The scene model is available in the \scenes directory of the simAPI path. Using the sliders or input fields in the GUI, attempt to bring the robotic hand to reach one of the objects, iteratively adjusting the Arm Joints one at a time. Repeat until you reach the object and then close the fingers over it, as if you were to grab the object. Observe the relative complexity of performing this apparently simple task with a manipulator robot and joystick control only.

   Note: If at any point during the simulation the robot collides with itself or the floor, the simulation could become unstable due to how the physics engine works. It is then recommended to restart the simulation from scratch.

7. Close the GUI by closing the related window and move on to Section 7.4.

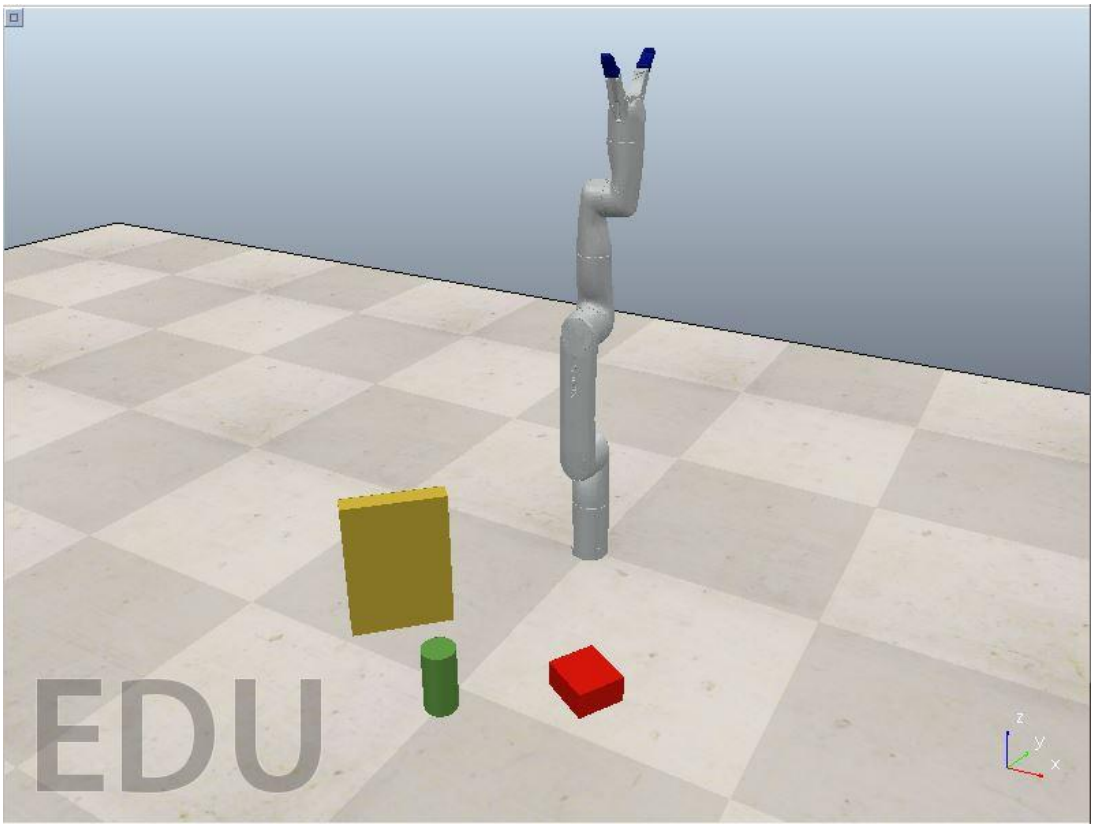


Figure 16: Scene with Gen3Lite model and primitive object shapes (`Gen3Lite blocks.ttt`)

7.4 Motion commands operation

This section explains how to operate the arm joints and fingers of the Gen3 lite robot using motion commands that should be more precise than the sliders available on the GUI. These commands also allow writing more elaborate script programs.

1. Create a VREP instance and start communication with the simulator
2. Load a robot model
3. Start simulation
4. Create a Gen3Lite instance
5. Connect to the robot
6. **Set angular joint values**
7. **Get current joint values**
8. Disconnect the robot
9. Stop simulation
10. Close the scene
11. End communication with simulator and destroy the VREP instance

The MATLAB program that runs this example is detailed here.

```
v = VREP('path','write CoppeliaSim path here')
v.loadmodel('Gen3Lite robot elevated','local',true)
v.simstart()
j = GEN3LITE(v,'Gen3Lite')
j.connect()
j.sendJointAndFingerPositionCommand([3*pi/4,pi/4,pi/4,0,-pi/4,0],[0])
q = j.getJointAndFingerPos()
j.disconnect()
v.simstop()
v.clearscene()
v.delete()
```

This program is based on the one in Section 7.1. The new steps (6-7) are discussed in more detail below.

7.4.1 Set angular joint values and operate the gripper

Instead of relying on the GUI to make the robot joints move to desired angles, you can call the `sendJointAndFingerPositionCommand()` method on the Gen3Lite instance `j` in a MATLAB script. This method expects as arguments a target angle for every arm joint in the form of an array of six angles (in radians), and a second array with a single value contained between 0 and 100 that controls the percentage of closure of the two fingers (0 opens the gripper, while 100 closes the gripper fully).

```
j.sendJointAndFingerPositionCommand([3*pi/4,pi/4,pi/4,0,-pi/4,0],[0])
```

In a case where you only want to set the arm joints, use the `sendJointPositionCommand()` method, or use the `sendFingerPositionCommand()` method to set the gripper closure. Alternatively, you can also use the `goToHomePosition()` method that is very helpful to bring the entire robot back to its "Home Configuration" without needing to specify any of the arm joints or fingers angle values. This method plays a similar role to the `Rehome` button in the GUI.

Go to CoppeliaSim and you should see each of the joints moving until they reach the target values. The configuration of the robot should then look like the one shown in Figure 17 depending on the observation point selected in CoppeliaSim.

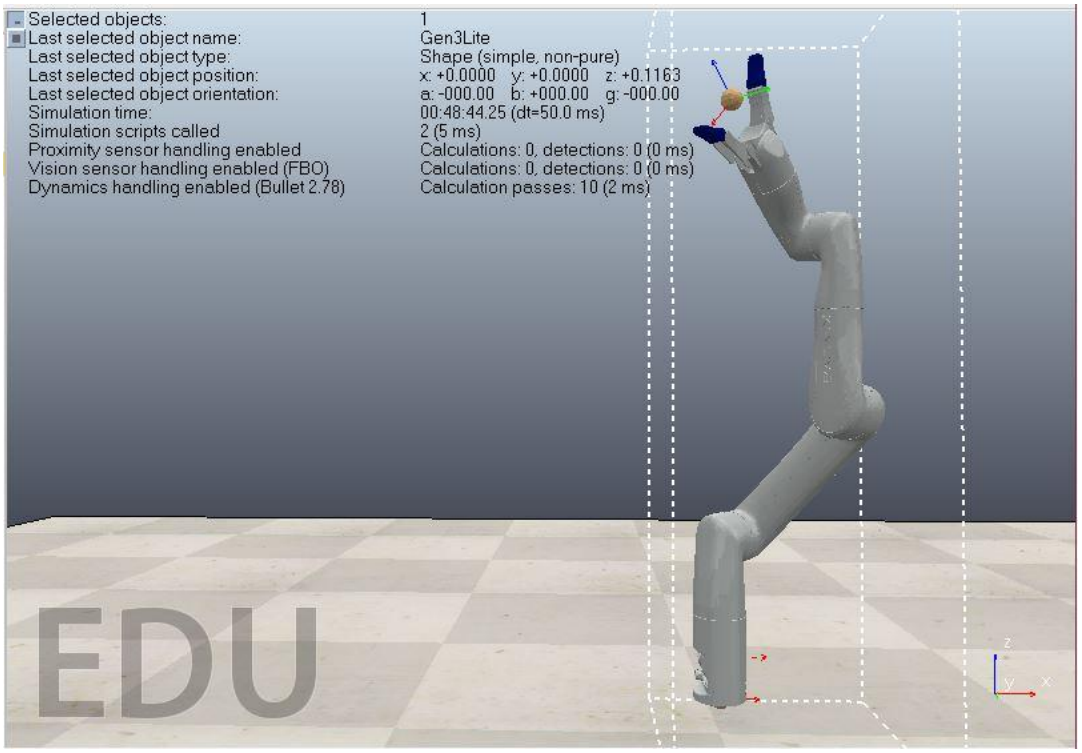


Figure 17: Robot configuration for the target joint values (`[3*pi/4,pi/4,pi/4,0,-pi/4,0],[0]`)

7.4.2 Get current joint values

To obtain the current values of the robot joints, you can use the `getJointAndFingerPos()` method on the Gen3Lite instance `j`. This method returns the values of the arm joints in an array of angles (in radians) followed by a percentage value for the fingers' percentage of closure.

```
q = j.getJointAndFingerPos()
```

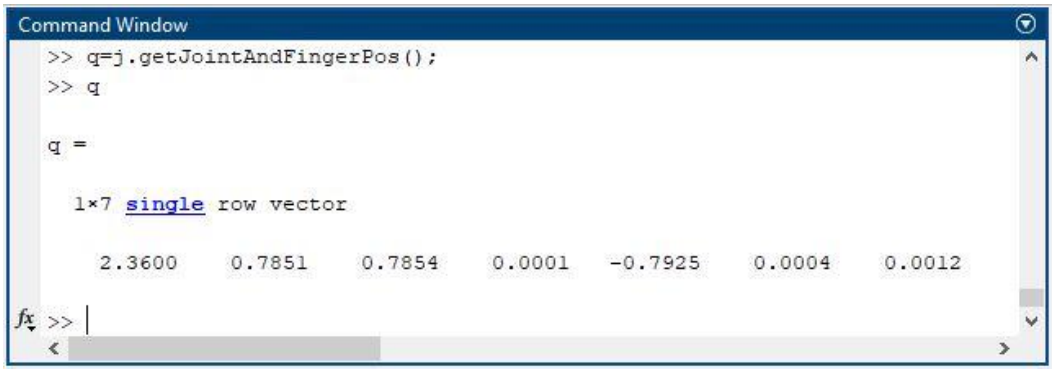


Figure 18: Accessing combined arm and finger joints values

Alternatively, you can consult the properties `JointPos` and `FingerPos` of the Gen3Lite instance `j`. These properties return the same output as the method

`getJointAndFingerPos()`, but arm and hand joints data are accessed separately. Use whichever is more appropriate based on the task that you are implementing.

```
j.JointPos
j.FingerPos
```

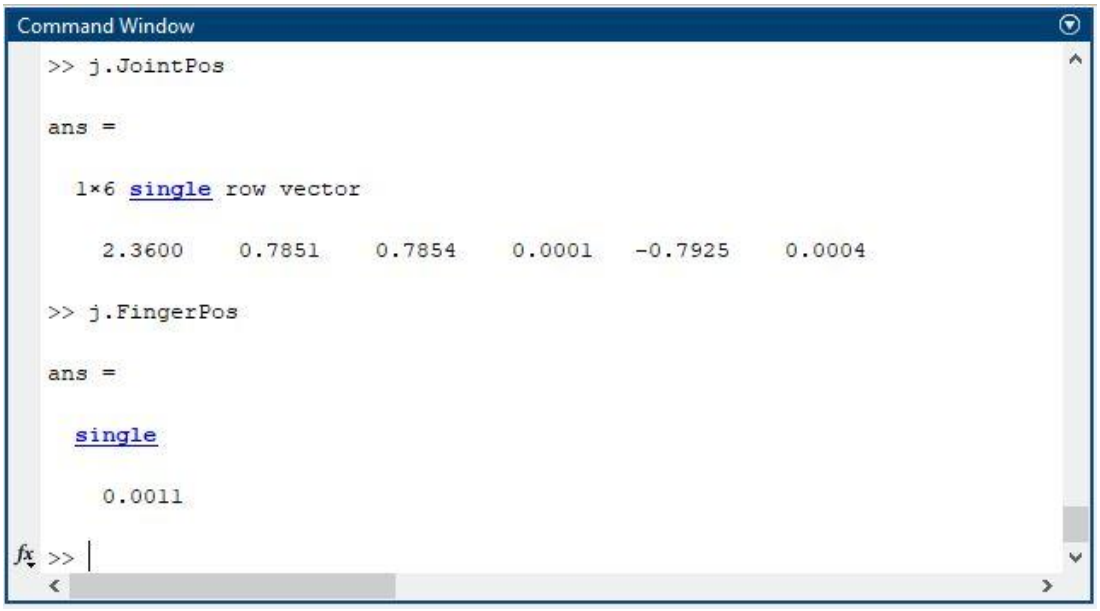


Figure 19: Accessing separate arm and finger joints values

7.4.3 Experimental activities with motion commands

Perform the following activities to become familiar with programming the operation of the Gen3 lite robot and the control of its movements. The scripting method, unlike the GUI, provides a more flexible and systematic way to control the robot motion, improve repeatability, and avoid requiring skilled operators. Programming a robot proves especially helpful when dealing with robot architectures containing a large number of degrees of freedom.

1. Using the `Gen3Lite robot elevated` model, repeat activities #1 to #5 from Section 7.3.2. Use Joint and Finger Position commands to assign joint angles instead of the GUI, by programming a MATLAB script to control the movement of the simulated Gen3 lite robot. Observe that simultaneous movement of all joints can be achieved this way, expediting the reconfiguration of the manipulator.

2. Replace the `Gen3Lite robot elevated` model with the `Gen3Lite blocks` scene. Repeat Activity #6 from Section 7.3.2 where you attempted to reach out to a virtual object with the goal to grab it using the robotic hand. But this time use Joint and Finger Position commands instead of the GUI. Observe again the relative complexity of performing this apparently simple task while not benefitting from meticulous planning of the sequence of movements to be executed by the robot. This will help you understand the motivation behind

developing formal kinematic models for manipulator robots, as will be studied in an upcoming part of this course.

3. Once you will have closely observed how motion commands drive the manipulator, define yourself a pick-and-place operation of your choice inspired by those shown during lectures (e.g., picking one of the objects and depositing it in a new location). Be inventive and propose a different task (e.g., pick another object, stack some objects up, relocate objects elsewhere, etc). Diversity among tasks proposed by the teams is expected. Develop a MATLAB script using motion commands as defined above to automate the chosen task. Determine by trial-and-error the important joint angular values that the robot must visit in order to perform the task. Finally, record a short video (max. 1 minute) of the CoppeliaSim display as the simulated robot performs the task you selected (any screen recording tool or cellphone will do).

   Make sure to save this video and your MATLAB script on a personal external media before logging off from the workstation at STE-3040. You will need to submit the video and script as part of your lab evaluation.

4. Proceed to the laboratory evaluation in Section 8.

5. Properly disconnect the simulated robot, stop the simulation, clear the `Gen3Lite blocks` scene, end the communication with simulator and destroy the VREP instance. Close the MATLAB and CoppeliaSim applications and disconnect from the workstation at STE-3040 after having saved your MATLAB scripts and video on an external media.

8 Laboratory experimental activities evaluation

To complete this first laboratory, proceed with the online evaluation that is proposed on Brightspace, under Quizzes, Lab 1 Evaluation. This evaluation must be performed **individually for every team** and **submitted at the latest by Friday, Oct. 9th, 2026 at 11:59PM**.

Perform a pick-and-place task of your choice as defined in Section 7.4.3.3, using script motion commands. Prepare your `.m` file containing your MATLAB script with the sequence of joint and finger configurations visited by the robot. Prepare also your video file. Then, go on Brightspace and complete the "**Lab 1 Evaluation**" quiz.

There is no formal lab report to submit for this lab beyond what is asked for on Brightspace.